Execution Context

Global, Function & Eval — The Complete Deep Dive

Mental Model, Internals, Code Examples, Gotchas, System Design, Interview Questions & Cheat Sheet

Prepared by Claude for Akshay

Table of Contents

1. Mental Model — The "Why"

JavaScript needs a system to answer these questions while running your code: **What variables exist right now? What's the value of this? Which code is currently running? What's the scope chain?** The Execution Context is JavaScript's **workspace** — it sets up everything a piece of code needs before it runs.

> ■ **The Chef's Kitchen Analogy**
> Imagine a restaurant kitchen. Each order (function call) gets a **prep station** with:
> **Ingredients list** = variables and function declarations (Variable Environment)
> **Recipe card** = the actual code to execute
> **Chef assigned** = what `this` refers to
> **Access to pantry** = scope chain (can access shared/global ingredients)
>
> When a new order comes in, a new station is set up. When done, the station is cleaned up. Multiple orders stack up — the chef works on the top one first (call stack).

Where It Sits in the Ecosystem

Related concepts that depend on Execution Context: Scope Chain, Hoisting, Closures, `this` keyword, Call Stack, Event Loop — they ALL connect back here.

2. How It Actually Works — Internals

The Three Types of Execution Context

Type	Created When	How Many	Key Detail
Global EC (GEC)	Script first loads	Exactly 1 per program	Creates window/global, sets this
Function EC (FEC)	Function is called	One per call	Each call = new context
Eval EC	eval() runs	One per eval	Avoid! Security risk

Two Phases of Every Execution Context

This is the most important concept — every EC goes through **two phases**:

What Gets Set Up in Creation Phase

Declaration	Creation Phase Sets To	Execution Phase Sets To
var x = 5	x = undefined	x = 5

let y = 10	y = (uninitialized)	y = 10
const z = 15	z = (uninitialized)	z = 15
function foo(){}	foo = full function body	(already complete)
var bar = ()=>{}	bar = undefined	bar = arrow function
this	Determined now	(already set)
scope chain	Linked to parent	(already set)

3. Code Deep-Dive — Tracing Through Real Code

Let's trace through this code step by step:

```
var name = "Akshay";
let age = 25;
function greet(message) {
  var prefix = "Hello";
  console.log(`${prefix} ${name}, ${message}`);
}
greet("welcome!");
```

Step 1: Global EC — CREATION PHASE

```
Global Execution Context (Creation):
+-------------------------------------+
| Variable Environment:               |
|   name         -> undefined (var)   |
|   age          -> <uninitialized>   |
|   greet        -> function() {...}  |
|                                     |
| Scope Chain:   [Global]             |
| this:          window (browser)     |
+-------------------------------------+
```

Step 2: Global EC — EXECUTION PHASE

```
Line 1: name = "Akshay"     -> name is now "Akshay"
Line 2: age = 25              -> age is now 25
Line 4-7: skip (already hoisted)
Line 9: greet("welcome!")    -> CREATE new Function EC
```

Step 3: greet() Function EC — CREATION PHASE

```
greet() Execution Context (Creation):
+-------------------------------------+
| Variable Environment:               |
|   arguments -> { 0: "welcome!" }    |
|   message   -> "welcome!"           |
|   prefix    -> undefined (var)      |
|                                     |
| Scope Chain: [greet, Global]        |
| this:        window (regular call) |
+-------------------------------------+
```

Step 4: greet() Function EC — EXECUTION PHASE

```
Line 5: prefix = "Hello"
Line 6: console.log(...)
```

```
   -> needs 'prefix': found in own scope
   -> needs 'name': not here, goes up -> found in Global
   -> needs 'message': found in own scope (parameter)
   -> prints "Hello Akshay, welcome!"
```

Step 5: greet() finishes -> its EC is POPPED off the call stack

Call Stack Visualization

```
STEP 1:              STEP 2:              STEP 3:
+-----------+       +-----------+       +-----------+
|           |       | greet() EC|       |           |
|           |       +-----------+       |           |
| Global EC |       | Global EC |       | Global EC |
+-----------+       +-----------+       +-----------+
 Script starts       greet() called      greet() done
                                         (popped off)
```

Real-World Example — Nested Contexts

```
const API_BASE = "https://api.example.com"; // Global EC
async function fetchUserData(userId) {
  // New Function EC created
  // Scope chain: [fetchUserData, Global]
  // Has access to API_BASE via scope chain
  const url = `${API_BASE}/users/${userId}`;
  try {
    const response = await fetch(url);
    const data = await response.json();
    // Inner function = ANOTHER EC + closure
    const formatUser = (fields) => {
      // Scope: [formatUser, fetchUserData, Global]
      return fields.reduce((result, field) => {
        result[field] = data[field];
        return result;
      }, { id: userId });
    };
    return formatUser(["name", "email", "role"]);
  } catch (error) {
    console.error(`Failed: ${error.message}`);
    return null;
  }
}
```

✔ Key Observation

Each function call creates a new EC. The inner `formatUser` forms a **closure** over `data` and `userId` — those variables survive even after fetchUserData's EC is popped, because the closure holds a reference.

4. Gotchas & Tricky Parts

Gotcha 1: var vs let vs function — Different Hoisting

```
// var: hoisted as undefined (no error, just weird)
console.log(x);  // undefined
var x = 10;
// let: hoisted but TDZ (crashes!)
console.log(y);  // ReferenceError!
let y = 10;
// function declaration: fully hoisted (works!)
greet();  // "Hello!"
function greet() { console.log("Hello!"); }
// function EXPRESSION: follows var/let rules
sayHi();  // TypeError: sayHi is not a function
var sayHi = function() { console.log("Hi!"); };
// sayHi is hoisted as undefined, not as function!
```

■ Why It Breaks

In Creation Phase: `var` gets `undefined`, `let/const` stays uninitialized (TDZ), function declarations get the full body, function expressions follow their declaration keyword's rules.

Gotcha 2: Each Function CALL = NEW Context

```
// WRONG: thinking same function = same context
function counter() {
  var count = 0;
  count++;
  console.log(count);
}
counter();  // 1
counter();  // 1 (NOT 2!)
counter();  // 1 (NOT 3!)
// Each call = fresh EC with count = 0
// FIX: use closure to persist state
function createCounter() {
  var count = 0;
  return function() { count++; console.log(count); };
}
const increment = createCounter();
increment();  // 1
increment();  // 2
increment();  // 3
```

Gotcha 3: 'this' Is Set During EC Creation

```
const user = {
  name: "Akshay",
  greet() { console.log(this.name); }
```

```
};
// Extracting the method changes EC's 'this'
const fn = user.greet;
fn();           // undefined! (no dot = no this)
user.greet();   // "Akshay" (dot rule = this = user)
```

Gotcha 4: var Inside Blocks Doesn't Create Block Scope

```
// var leaks out of blocks
if (true) { var x = 10; }
console.log(x);  // 10 (leaked!)
for (var i = 0; i < 3; i++) {}
console.log(i);  // 3 (leaked!)
// let/const create proper block scope
if (true) { let y = 10; }
console.log(y);  // ReferenceError!
```

■ Why

`var` is stored in the EC's Variable Environment (function-scoped). `let/const` use the Lexical Environment which respects block boundaries.

Gotcha 5: arguments Object Quirks

```
function test(a, b) {
  console.log(arguments);    // [1, 2, 3, 4] ALL args
  console.log(arguments[2]); // 3 (extra args accessible)
}
test(1, 2, 3, 4);
// arguments is NOT a real array!
function broken() {
  arguments.map(x => x * 2);  // TypeError!
}
// FIX: use rest params
function fixed(...args) {
  args.map(x => x * 2);  // works! real array
}
// Arrow functions DON'T have arguments!
const arrow = () => {
  console.log(arguments);  // ReferenceError!
};
```

Gotcha 6: eval() Is Dangerous

```
var secret = "password123";
// eval creates EC that accesses parent scope!
eval("console.log(secret)");  // "password123"
eval("secret = 'hacked'");    // modifies parent!
console.log(secret);           // "hacked"
// Safer alternative (if you must):
const fn = new Function("return 42");
fn();  // 42 (runs in global scope, can't access locals)
```

■ Never Use eval()

It creates an EC that shares the caller's scope chain — massive security risk, kills engine optimizations, makes debugging impossible.

Gotcha 7: The Classic Loop Closure Bug

```
// BUG: prints 3, 3, 3 (not 0, 1, 2)
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
// Only ONE 'i' in the EC (var is function-scoped)
// By timeout time, loop done, i = 3
// FIX: use let (block-scoped, new i each iteration)
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
// Prints: 0, 1, 2
```

5. Comparison — The Three EC Types

Feature	Global EC	Function EC	Eval EC
Created when	Script loads	Function is called	eval() runs
How many	Exactly 1	One per call	One per eval
this value	window / global	Depends on call	Inherits from caller
Has arguments?	No	Yes (regular fn)	Yes
Variables go where	Global object	Local scope	Caller's scope
Destroyed when	Page closes	Function returns	eval() finishes
Performance	Minimal impact	Normal	Heavy (deoptimizes)
Use in production	Always exists	Primary mechanism	NEVER use

How Engines Handle Variable Storage

Approach	Mechanism	Applies To	Hoisting Behavior
Variable Environment	Stored in EC's var env	var, function decl	Hoisted + initialized
Lexical Environment	Separate from var env	let, const	Hoisted but TDZ
Closure Scope	Kept in heap memory	Captured variables	Survives EC destruction
Block Scope	Mini-scope within EC	let/const in { }	Scoped to block only

6. System Design — Performance at Scale

Call Stack Depth — Stack Overflow

```
// Each recursive call = new EC on the stack
function fibonacci(n) {
  if (n <= 1) return n;
  return fibonacci(n-1) + fibonacci(n-2);
}
fibonacci(10000);  // RangeError: Maximum call stack size!
// FIX: iterative approach = only 1 Function EC
function fibIterative(n) {
  let a = 0, b = 1;
  for (let i = 2; i <= n; i++) [a, b] = [b, a + b];
  return b;
}
```

Closure Memory Implications

```
// BAD: closure holds ENTIRE parent scope
function processData() {
  const hugeArray = new Array(1000000).fill("data");
  return function() {
    console.log(hugeArray[0]);
    // hugeArray stays in memory because of closure!
  };
}
// GOOD: capture only what you need
function processDataFixed() {
  const hugeArray = new Array(1000000).fill("data");
  const firstItem = hugeArray[0]; // extract needed data
  return function() {
    console.log(firstItem);
    // hugeArray can be garbage collected now
  };
}
```

Performance Considerations

Factor	Impact	Mitigation
Deep recursion	Stack overflow	Use iteration or trampoline
Closures over large scopes	Memory leak	Capture only needed vars
eval()	Deoptimizes entire function	Never use eval()

with statement	Prevents scope optimization	Never use with
Function creation in loops	Extra GC pressure	Define functions outside
Excessive nesting	Long scope chains	Flatten where possible

7. Interview-Ready Questions

★ Junior Level
Conceptual understanding

Q1: What are the two phases of an Execution Context?

A: Creation Phase — JS sets up Variable Environment (hoists var as undefined, stores function declarations fully, marks let/const as uninitialized), creates Scope Chain, determines `this`. **Execution Phase** — code runs line by line, variables get actual values.

Q2: Why does `console.log(x)` print undefined instead of 10?

```
console.log(x);
var x = 10;
```

A: During Creation Phase, `var x` is hoisted and initialized to `undefined`. The assignment `x = 10` only happens during Execution Phase when that line is reached.

Q3: What is the Global Execution Context?

A: The default EC created when your script loads. Only one per program. Creates the global object (window/global), sets `this` to it, and top-level `var` and function declarations become properties of the global object.

★ Mid Level
Implementation + edge cases

Q4: What's the output?

```
var a = 1;
function outer() {
  console.log(a);  // ?
  var a = 2;
  console.log(a);  // ?
}
outer();
```

A: `undefined`, then 2. The local `var a` is hoisted inside outer()'s EC and shadows the global a. First log sees hoisted `undefined`, second sees 2 after assignment.

Q5: Why does var in a loop cause closure bugs?

A: `var` is function-scoped, so there's only ONE `i` in the EC shared by all callbacks. When timeouts fire, the loop is done and `i` has its final value. `let` fixes this by creating a new binding per iteration.

Q6: What happens to a Function EC after the function returns?

A: It's popped off the call stack and eligible for GC — unless a closure exists. If an inner function references parent variables, those are kept alive in heap memory via the closure scope.

Q7: Why is eval() problematic from an EC perspective?

A: eval() creates an EC with access to the caller's scope chain. The engine can't optimize variable access at compile time (doesn't know what eval might read/modify), forcing slower dictionary-based lookups instead of fast indexed access.

★ Senior Level
System design + optimization

Q8: How would you debug "Maximum call stack size exceeded" in a React + Next.js app?

A: Each function call adds an EC to the stack (~10-15K frame limit). Usually means infinite recursion: component re-rendering itself, circular useEffect deps, or missing base case. Debug: check stack trace for repeating functions, use React DevTools for render loops, add console.count to suspects, verify useEffect dependency arrays.

Q9: How does V8 optimize ECs and what breaks those optimizations?

A: V8 uses hidden classes and inline caching for fast property access. For ECs, it pre-computes variable positions for index-based access. Things that break this: eval() forces dynamic scope, with statements inject dynamic scope, delete on variables changes hidden class, arguments object mutations. V8 "deoptimizes" — falls back to interpreted mode.

Q10: Design a system to safely execute untrusted user code in Node.js.

A: Key: isolate the EC's scope chain. Use `vm.createContext()` + `vm.runInContext()` for a sandboxed Global EC with custom global object. For stronger isolation, use `worker_threads` or `isolated-vm` (separate V8 isolate with own heap). Never use eval() or new Function() — they share the caller's scope.

8. Cheat Sheet — Quick Reference

EC Creation — What Gets Set Up

```
CREATION PHASE (before code runs):
+--------------------------------------+
| var x = 5;        ->  x = undefined    |
| let y = 10;       ->  y = <TDZ>        |
| const z = 15;     ->  z = <TDZ>        |
| function foo(){}  ->  foo = f foo(){} |
| var bar = ()=>{}  ->  bar = undefined  |
| this              ->  determined now   |
| scope chain       ->  linked to parent |
+--------------------------------------+
EXECUTION PHASE (code runs line by line):
  x = 5, y = 10, z = 15, bar = ()=>{}
```

Common Patterns

```
// IIFE - creates and immediately destroys an EC
(function() {
  var private = "hidden";
})();
// Module Pattern - closure + EC for private vars
const module = (function() {
  let count = 0;
  return {
    increment: () => ++count,
    getCount: () => count,
  };
})();
// Avoid global pollution - wrap in module
const App = (() => {
  const name = "Akshay";
  return { name };
})();
```

Do's and Don'ts

✔ DO

Use `let/const` (block-scoped, predictable). Minimize global variables. Understand hoisting (Creation Phase). Keep closure scopes small. Use strict mode. Prefer iteration over deep recursion.

■ **DON'T**

Use `eval()` (dangerous EC, kills optimization). Use `with` statement (messes with scope chain). Declare functions inside blocks with var. Rely on hoisting for readability. Create closures over huge data accidentally. Use `var` in loops expecting block scope.

One-Liner Tips

■ **Quick Rules**

"Two phases" = Creation (setup) + Execution (run)

`var` hoists with undefined, `let/const` hoist with TDZ

Function declarations hoist fully, expressions don't

Every function CALL = new EC, new variables, new `this`

Closures = EC variables surviving after EC is popped

Call stack = pile of ECs, currently running on top

Global EC = bottom of stack, destroyed when page closes

Arrow functions inherit `this` and `arguments` from parent EC

9. Mini-Projects to Solidify Understanding

★ Project 1: Call Stack Visualizer
Create a React app where you paste JS code and it visually shows ECs being pushed/popped on the call stack. Show the Variable Environment contents for each EC. This forces you to mentally simulate what the engine does.

★ Project 2: Mini JavaScript Interpreter
Build a function that takes JavaScript-like code strings and executes them, creating your own EC objects with `variableEnvironment`, `scopeChain`, and `thisBinding`. Handle var, let, function declarations, and nested functions. This is the deepest way to understand ECs.

★ Project 3: Closure Memory Leak Detector
Build a Node.js tool that instruments functions to track which variables are captured in closures and how much memory they hold. Use `process.memoryUsage()` to compare before/after. This teaches you the practical consequences of EC lifetimes and closure scope chains.

You now understand how JavaScript ACTUALLY runs your code. Every variable lookup, every `this` confusion, every hoisting quirk — it all comes back to Execution Contexts.

Next up: Promises & async/await — how JavaScript handles asynchronous code with the Event Loop.